

Metadatos ICFES

icfes: competencia: - formulacion_ejecucion nivel_dificultad: 4 contenido: categoria: estadistica tipo:
no_generico contexto: cotidiano eje_axial: eje3 componente: aleatorio

```
options(OutDec = ".") # Asegurar punto decimal en este chunk
# Establecer semilla aleatoria para reproducibilidad
set.seed(sample(1:10000, 1))
# Aleatorizar deportes para el contexto del problema
deportes <- c("fútbol", "baloncesto", "voleibol", "hockey", "balonmano",
             "rugby", "waterpolo", "béisbol", "futsal", "handball")
deporte <- sample(deportes, 1)
# Aleatorizar nombres de competiciones
competiciones <- c("campeonato", "torneo", "liga", "copa", "certamen",
                  "competición", "mundial", "clasificatoria", "eliminatória")
competicion <- sample(competiciones, 1)
# Aleatorizar fases de competición
fases_inicial <- c("primera ronda", "fase inicial", "fase de grupos",
                  "etapa preliminar", "fase clasificatoria")
fase_inicial <- sample(fases_inicial, 1)
fases_siguiete <- c("siguiete ronda", "segunda fase", "siguiete etapa",
                  "fase final", "etapa eliminatória", "siguiete instancia")
fase_siguiete <- sample(fases_siguiete, 1)
# Aleatorizar puntos necesarios (entre 20 y 40, números que sean alcanzables)
puntos_necesarios <- sample(c(20:40), 1)
# Aleatorizar cantidad de partidos (entre 8 y 16)
partidos_por_equipo <- sample(8:16, 1)
# Aleatorizar sistema de puntuación (manteniendo estructura realista)
# Opciones para puntos por ganar (2 o 3 son los más comunes)
puntos_victoria <- sample(c(2, 3), 1, prob = c(0.3, 0.7)) # 3 es más común
# Puntos por empate (usualmente 1, pero podría ser otro valor)
puntos_empate <- sample(c(1, 2), 1, prob = c(0.9, 0.1)) # 1 es mucho más común
# Puntos por derrota (usualmente 0, pero podría ser otro valor)
puntos_derrota <- sample(c(0, 1), 1, prob = c(0.95, 0.05)) # 0 es mucho más común
# Verificar que las combinaciones sean realistas
if (puntos_victoria <= puntos_empate) {
  puntos_victoria <- puntos_empate + 1
}
if (puntos_empate <= puntos_derrota) {
  puntos_empate <- puntos_derrota + 1
}
# Verificar que los puntos necesarios sean alcanzables y desafiantes
max_puntos_posibles <- puntos_victoria * partidos_por_equipo
min_puntos_posibles <- puntos_derrota * partidos_por_equipo
# Ajustar los puntos necesarios si están fuera de rango
if (puntos_necesarios > max_puntos_posibles * 0.9) {
  puntos_necesarios <- round(max_puntos_posibles * 0.8)
}
if (puntos_necesarios < min_puntos_posibles + 1) {
  puntos_necesarios <- min_puntos_posibles + round(partidos_por_equipo / 2)
}
# Cálculo para la respuesta correcta:
# ¿Cuál es el mínimo de partidos que se deben ganar para clasificar?
calcular_combinaciones <- function(total_partidos, puntos_objetivo, pts_victoria, pts_empate, pts_derrota)
```

```

combinaciones <- list()
for (victorias in 0:total_partidos) {
  for (empates in 0:(total_partidos - victorias)) {
    derrotas <- total_partidos - victorias - empates
    puntos_totales <- victorias * pts_victoria + empates * pts_empate + derrotas * pts_derrota
    if (puntos_totales >= puntos_objetivo) {
      combinaciones[[length(combinaciones) + 1]] <- list(
        victorias = victorias,
        empates = empates,
        derrotas = derrotas,
        puntos = puntos_totales
      )
    }
  }
}
return(combinaciones)
}

# Obtener todas las combinaciones que alcanzan o superan los puntos necesarios
combinaciones_validas <- calcular_combinaciones(
  partidos_por_equipo,
  puntos_necesarios,
  puntos_victoria,
  puntos_empate,
  puntos_derrota
)

# Ordenar combinaciones por número de victorias (ascendente)
combinaciones_ordenadas <- combinaciones_validas[order(
  sapply(combinaciones_validas, function(x) x$victorias),
  sapply(combinaciones_validas, function(x) x$empates),
  decreasing = FALSE
)]

# El mínimo de partidos a ganar será la primera combinación ordenada
min_victorias <- if (length(combinaciones_ordenadas) > 0) {
  combinaciones_ordenadas[[1]]$victorias
} else {
  NA # Si no hay combinaciones válidas
}

min_empates <- if (length(combinaciones_ordenadas) > 0) {
  combinaciones_ordenadas[[1]]$empates
} else {
  NA
}

min_derrotas <- if (length(combinaciones_ordenadas) > 0) {
  combinaciones_ordenadas[[1]]$derrotas
} else {
  NA
}

# Verificar que la solución sea correcta y ajustar si no lo es
if (is.na(min_victorias) ||
  (min_victorias * puntos_victoria + min_empates * puntos_empate +
   min_derrotas * puntos_derrota) < puntos_necesarios) {
  # Recalcular puntos necesarios para garantizar solución
  puntos_necesarios <- (partidos_por_equipo %/% 2) * puntos_victoria +

```

```

                                (partidos_por_equipo %/% 4) * puntos_empate
# Recalcular combinaciones
combinaciones_validas <- calcular_combinaciones(
  partidos_por_equipo,
  puntos_necesarios,
  puntos_victoria,
  puntos_empate,
  puntos_derrota
)
combinaciones_ordenadas <- combinaciones_validas[order(
  sapply(combinaciones_validas, function(x) x$victorias),
  sapply(combinaciones_validas, function(x) x$empates),
  decreasing = FALSE
)]
min_victorias <- combinaciones_ordenadas[[1]]$victorias
min_empates <- combinaciones_ordenadas[[1]]$empates
min_derrotas <- combinaciones_ordenadas[[1]]$derrotas
}

# Crear texto explicativo para la solución
explicacion_victorias <- paste("Con", min_victorias, "victorias,", min_empates,
                              "empates y", min_derrotas, "derrotas, se obtienen",
                              min_victorias * puntos_victoria + min_empates * puntos_empate + min_derrotas * puntos_derrota,
                              "puntos, lo que es suficiente para clasificar.")

# Crear algunas explicaciones alternativas con otras combinaciones posibles
explicaciones_alt <- c()
if (length(combinaciones_ordenadas) > 1) {
  # Tomar hasta 3 combinaciones adicionales para mostrar como ejemplos
  for (i in 2:min(4, length(combinaciones_ordenadas))) {
    comb <- combinaciones_ordenadas[[i]]
    puntos_total <- comb$victorias * puntos_victoria + comb$empates * puntos_empate + comb$derrotas * puntos_derrota
    explicaciones_alt <- c(explicaciones_alt,
                          paste("Otra posibilidad es:", comb$victorias, "victorias,",
                                comb$empates, "empates y", comb$derrotas, "derrotas, para un total de",
                                puntos_total, "puntos."))
  }
}

# Generar opciones de respuesta y definir la correcta
opciones <- c(
  paste("¿Cuál es el mínimo de partidos que se deben ganar para clasificar?"),
  paste("¿Cuál es la relación entre victorias y empates que minimiza el número total de victorias neces"),
  paste("¿Cuál es la distribución óptima de resultados que maximiza la probabilidad de clasificación con"),
  paste("¿Cuál es la estrategia de juego que garantiza la clasificación con la menor cantidad de victor")
)

# La primera opción es la correcta
opcion_correcta <- 1

# Vector de solución para r-exams
solucion <- c(1, 0, 0, 0)

# Colores para la tabla TikZ
colores_disponibles <- c("red", "blue", "green", "orange", "purple", "teal", "brown", "gray")
color_fondo_header <- sample(colores_disponibles, 1)
intensidad_header <- sample(c(15, 20, 25, 30), 1)
color_header <- paste0(color_fondo_header, "!", intensidad_header)
color_fondo_filas <- sample(colores_disponibles[colores_disponibles != color_fondo_header], 1)

```

```

intensidad_filas <- sample(c(10, 15, 20), 1)
color_filas <- paste0(color_fondo_filas, "!", intensidad_filas)

options(OutDec = ".") # Asegurar punto decimal en este chunk
# Función para generar el código TikZ de la tabla
generar_tabla_puntuacion <- function(competicion, deporte, puntos_victoria, puntos_empate, puntos_derrota) {
  # Crear tabla con TikZ
  tabla_code <- c("\\begin{tikzpicture}",
    "\\node[inner sep=0pt] {",
    "  \\begin{tabular}{|c|c|}",
    "    \\hline",
    paste0("      \\rowcolor{", color_header, "}"),
    paste0("      \\multicolumn{2}{|c|}{\\textbf{Sistema de puntuación}} \\\\"),
    "    \\hline",
    paste0("      \\rowcolor{", color_filas, "}"),
    paste0("      Partido ganado & ", puntos_victoria, " puntos \\\\"),
    "    \\hline",
    paste0("      Partido empatado & ", puntos_empate, " punto", if(puntos_empate != 1) "s" else "", " \\\\"),
    "    \\hline",
    paste0("      Partido perdido & ", puntos_derrota, " punto", if(puntos_derrota != 1) "s" else "", " \\\\"),
    "    \\hline",
    "  \\end{tabular}",
    "};",
    "\\end{tikzpicture}")
  return(tabla_code)
}

# Generar código TikZ para la tabla
tabla_puntuacion <- generar_tabla_puntuacion(competicion, deporte, puntos_victoria, puntos_empate, puntos_derrota)

# Función simplificada para mostrar combinaciones de resultados
generar_tabla_ejemplos <- function(min_victorias, min_empates, min_derrotas, puntos_victoria, puntos_empate, puntos_derrota) {
  puntos_total <- min_victorias * puntos_victoria + min_empates * puntos_empate + min_derrotas * puntos_derrota
  # Crear tabla con TikZ
  tabla_code <- c("\\begin{tikzpicture}",
    "\\node[inner sep=0pt] {",
    "  \\begin{tabular}{|c|c|c|c|}",
    "    \\hline",
    paste0("      \\rowcolor{", color_header, "}"),
    "      \\textbf{Victorias} & \\textbf{Empates} & \\textbf{Derrotas} & \\textbf{Puntos} \\\\"),
    "    \\hline",
    paste0("      \\rowcolor{", color_filas, "}"),
    paste0("      ", min_victorias, " & ", min_empates, " & ", min_derrotas, " & ", puntos_total, " \\\\"),
    "    \\hline",
    "  \\end{tabular}",
    "};",
    "\\end{tikzpicture}")
  return(tabla_code)
}

# Generar código TikZ para la tabla de ejemplo
tabla_ejemplo <- generar_tabla_ejemplos(
  min_victorias,
  min_empates,
  min_derrotas,
  puntos_victoria,
  puntos_empate,
  puntos_derrota)

```

```
puntos_derrota,
color_header,
color_filas
)
```

Question

En un certamen de waterpolo, un equipo está en fase clasificatoria y necesita 31 puntos para clasificar a la etapa eliminatoria. En la ronda actual cada equipo juega 13 partidos. Los puntos se reparten de la siguiente manera:

Sistema de puntuación	
Partido ganado	3 puntos
Partido empatado	1 punto
Partido perdido	0 puntos

¿Cuál de las siguientes preguntas se puede responder con la información dada?

Answerlist - ¿Cuál es el mínimo de partidos que se deben ganar para clasificar? - ¿Cuántos puntos debe obtener un equipo para asegurar su clasificación? - ¿Cuál es la diferencia mínima entre puntos obtenidos y puntos necesarios para clasificar? - ¿Qué porcentaje del total de partidos debe ganar un equipo para clasificar?

Solution

La respuesta correcta es: **¿Cuál es el mínimo de partidos que se deben ganar para clasificar?**

Con la información dada, podemos calcular cuál es el mínimo de partidos que un equipo debe ganar para alcanzar los 31 puntos necesarios para clasificar. Este problema puede formularse como un problema de programación lineal entera:

Planteamiento matemático:

Sea: - V = número de victorias - E = número de empates - D = número de derrotas - $P_v = 3$ (puntos por victoria) - $P_e = 1$ (puntos por empate) - $P_d = 0$ (puntos por derrota) - $P_{total} = 13$ (total de partidos) - $P_{necesarios} = 31$ (puntos necesarios para clasificar)

Queremos: - Minimizar: V (número de victorias) - Sujeto a: 1. $V \cdot P_v + E \cdot P_e + D \cdot P_d \geq P_{necesarios}$ (condición de puntos) 2. $V + E + D = P_{total}$ (restricción de partidos) 3. $V, E, D \geq 0$ y enteros (restricciones de no negatividad e integridad)

Este es un problema de optimización con variables enteras que puede resolverse mediante el método de enumeración completa, evaluando todas las combinaciones posibles de (V, E, D) que satisfagan las restricciones y seleccionando aquella con el menor valor de V .

Según el sistema de puntuación: - Victoria = 3 puntos - Empate = 1 punto - Derrota = 0 punto

La solución óptima que minimiza el número de victorias es:

Victorias	Empates	Derrotas	Puntos
9	4	0	31

Con **9 victorias**, 4 empates y 0 derrotas, se obtiene un total de 31 puntos, que es igual o superior a los 31 puntos necesarios para clasificar.

Verificación matemática de la solución óptima:

1. **Verificación de la restricción de partidos:** $V + E + D = P_{total}$ $9 + 4 + 0 = 13$ $13 = 13$ (Cumple)
2. **Verificación de la restricción de puntos:** $V \cdot P_v + E \cdot P_e + D \cdot P_d \geq P_{necesarios}$ $9 \cdot 3 + 4 \cdot 1 + 0 \cdot 0 \geq 31$ $27 + 4 + 0 = 31 \geq 31$ (Cumple)

3. **Demostración de optimalidad:** Para demostrar que esta solución es óptima, podríamos verificar que cualquier solución con menos victorias no cumple con la restricción de puntos. Si $V' = 8$, entonces el máximo de puntos posibles sería: $V' \cdot P_v + (P_{total} - V') \cdot P_e = 8 \cdot 3 + (13 - 8) \cdot 1 = 29$

Si este valor es menor que $P_{necesarios} = 31$, entonces la solución con $V = 9$ es efectivamente óptima.

Otra posibilidad es: 10 victorias, 1 empates y 2 derrotas, para un total de 31 puntos. Otra posibilidad es: 10 victorias, 2 empates y 1 derrotas, para un total de 32 puntos. Otra posibilidad es: 10 victorias, 3 empates y 0 derrotas, para un total de 33 puntos.

Con respecto a las otras opciones:

- B. **¿Cuál es la relación entre victorias y empates que minimiza el número total de victorias necesarias para clasificar?** Esta pregunta requiere un análisis más complejo que no puede resolverse solo con la información dada. Para determinar la relación óptima entre victorias y empates, necesitaríamos resolver un problema de optimización con restricciones donde la función objetivo sería minimizar el número de victorias sujeto a alcanzar al menos 31 puntos. Esto implicaría resolver:

Minimizar: V (número de victorias)

Sujeto a: $V \cdot P_v + E \cdot P_e + D \cdot P_d \geq P_{necesarios}$ y $V + E + D = P_{total}$

Donde P_v , P_e y P_d son los puntos por victoria, empate y derrota respectivamente.

Si bien podemos calcular una solución particular, no podemos determinar la relación matemática general sin información adicional sobre las restricciones tácticas del equipo.

- C. **¿Cuál es la distribución óptima de resultados que maximiza la probabilidad de clasificación con el mínimo esfuerzo?** Esta pregunta involucra conceptos de teoría de probabilidad y optimización que van más allá de la información proporcionada. Para responderla, necesitaríamos conocer:
 1. La distribución de probabilidad de ganar, empatar o perder cada partido
 2. La correlación entre el “esfuerzo” y la probabilidad de victoria
 3. La función que relaciona el esfuerzo con los resultados

Matemáticamente, esto requeriría resolver un problema de programación estocástica que no puede abordarse solo con los datos proporcionados.

- D. **¿Cuál es la estrategia de juego que garantiza la clasificación con la menor cantidad de victorias posibles?** Esta pregunta implica aspectos tácticos y estratégicos que no pueden determinarse únicamente con la información numérica proporcionada. Una estrategia de juego óptima dependería de:
 1. Las características de los oponentes
 2. El orden de los partidos
 3. La evolución del torneo

Desde el punto de vista matemático, esto requeriría un modelo de teoría de juegos secuenciales con información incompleta, que está fuera del alcance de los datos proporcionados.

Answerlist - Verdadero - Falso - Falso - Falso

Meta-information

exname: clasificacion_torneo_futbol_v2 extype: schoice exsolution: 1000 exshuffle: TRUE exsection: Análisis combinatorio|Álgebra|Sistemas de ecuaciones